ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

École Marocaine des Sciences de l'Ingénieur

(EMSI)

4e Année Informatique et Réseaux

Application de recettes

Rapport de Projet

Réalisé par :
Dounia Serbouti
El louadghiri El idrissi Douaa

Encadré par :
Mme naji zineb

Année universitaire : 2024 – 2025

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude à notre encadrante de projet, Mme Zineb NAJI, pour sa patience, son expertise technique et pédagogique, ainsi que pour ses conseils précieux qui ont guidé le développement de cette application. Son accompagnement a été déterminant dans la réussite de ce projet complexe.

Nos remerciements s'adressent également à l'ensemble du corps professoral de l'école EMSI, particulièrement au département informatique, pour nous avoir transmis les bases solides nécessaires à la compréhension des architectures logicielles modernes et du développement Full-Stack.

Enfin, nous remercions chaleureusement nos familles pour leur soutien inconditionnel et leurs encouragements constants durant toute la période de réalisation de ce travail technique.

Résumé

Le présent rapport documente la création de AppliRecette, une plateforme mobile innovante dédiée à la gastronomie communautaire. L'objectif principal était de concevoir un système capable de gérer de grands volumes de données culinaires tout en garantissant une expérience utilisateur fluide sur mobile.

L'architecture repose sur une pile technologique hybride : le frontend utilise React Native pour la flexibilité multiplateforme, tandis que le backend s'appuie sur une API RESTful en Node.js. La persistance des données est assurée par MongoDB, une base NoSQL choisie pour sa capacité à stocker des objets complexes comme des recettes. La dimension sécurité est gérée par Firebase Authentication, offrant une protection robuste des données personnelles.

L'application se distingue par son système de modération pré-publication, permettant à un administrateur de valider chaque contenu avant sa mise en ligne. Le résultat final est une solution complète, sécurisée et évolutive, répondant aux standards actuels du développement mobile.

Table des matières

Remerciements **1**

Résumé **2**

Introduction Générale **5**

1 Contexte, Analyse et Étude Technique **6**
1.1 Introduction du chapitre 6
1.2 Problématique et Besoins identifiés 6
1.3 Justification des choix technologiques 6
1.3.1 Sécurité et Authentification : Firebase 6
1.3.2 Le Backend : Node.js et l'API REST 6
1.3.3 Base de données NoSQL : MongoDB 7
1.3.4 Le Client Mobile : React Native 7
1.4 Conclusion du chapitre 7

2 Analyse Fonctionnelle et Conception UML **8**
2.1 Introduction du chapitre 8
2.2 Identification des Acteurs et Cas d'Utilisation 8
2.3 Modélisation des Données (Diagramme de Classe) 9
2.4 Dynamique du Système (Diagrammes de Séquence) 10
2.4.1 Diagramme de Séquence – Authentification 10
2.4.2 Diagramme de Séquence – Ajout d'une Recette 11
2.5 Diagramme d'Activité de l'Authentification 12
2.6 Conclusion du chapitre 13

3 Structure du Projet et Architecture Backend **14**
3.1 Introduction du chapitre 14
3.2 Organisation générale du code 14
3.2.1 Arborescence du Backend 14
3.3 La Logique des Contrôleurs 14
3.4 Le Modèle de Données Recette 14
3.5 Conclusion du chapitre 15

4 Réalisation des Interfaces et Étude des Modes Utilisateurs **16**
4.1 Introduction du chapitre 16
4.2 Le Mode Visiteur : Immersion Spontanée 16
4.3 Le Mode Utilisateur Authentifié : Interaction Sociale 17
4.3.1 L'Écran de Profil Personnel (Profil.js) 17
4.3.2 Système de Notifications en Temps Réel 19

4.3.3 Publication Dynamique et Signalements 19
4.4 Le Mode Administrateur : Tour de Contrôle 20
4.4.1 Dashboard et Statistiques 20
4.4.2 Gestion des Signalements et Validation 20
4.5 Conclusion du chapitre 22

5 Sécurité, Tests et Optimisations du Système 23
5.1 Introduction du chapitre 23
5.2 Stratégie de Sécurité Hybride 23
5.3 Tests de Fonctionnalité et Scénarios d'Erreurs 23
5.4 Optimisations de Performance Mobile 23
5.5 Conclusion du chapitre 24

Conclusion Générale et Perspectives 25

Table des figures

2.1 Diagramme de Cas d'Utilisation de l'application Mes Recettes 9
2.2 Diagramme de Classe représentant la structure des données 10
2.3 Diagramme de Séquence – Authentification via Firebase 11
2.4 Diagramme de Séquence – Ajout d'une recette 11
2.5 Diagramme d'Activité – Processus d'authentification 12

4.1 Interfaces du Mode Visiteur 17
4.2 Interface du Profil et Gestion Personnelle 18
4.3 Publication et Modération Communautaire 19
4.4 Interface d'Administration et Modération 21

Liste des tableaux

Introduction Générale

Le passage de l'analogique au numérique a radicalement transformé nos habitudes quotidiennes, et le domaine de la cuisine n'échappe pas à cette règle. Les livres de recettes traditionnels cèdent progressivement la place à des plateformes interactives et connectées. C'est dans ce contexte que s'inscrit le projet AppliRecette.

L'enjeu majeur de ce projet est de proposer une application mobile Full-Stack capable de fédérer une communauté de passionnés autour de la création culinaire. Il ne s'agit pas seulement de consulter des listes, mais de créer un véritable écosystème où chaque utilisateur peut publier ses propres créations, gérer ses recettes favorites et interagir avec les autres membres.

Ce rapport est structuré en cinq chapitres majeurs : l'analyse et l'étude technique, la phase de conception UML, l'organisation de l'architecture logicielle, la réalisation des interfaces utilisateurs, et enfin, la sécurité et l'optimisation du système.

Chapitre 1

Contexte, Analyse et Étude Technique

1.1 Introduction du chapitre

Avant de démarrer le développement technique, il est crucial d'analyser le marché et de définir les outils les plus adaptés. Ce chapitre explore la problématique du projet et justifie les choix technologiques qui constituent le socle de notre application.

1.2 Problématique et Besoins identifiés

La difficulté majeure pour les cuisiniers modernes est la dispersion des informations. Les recettes sont éparpillées entre différents supports. De plus, la qualité du contenu sur Internet est souvent inégale. AppliRecette répond à cela par un stockage centralisé et un contrôle administratif rigoureux. Les besoins identifiés sont la fluidité, la sécurité et la capacité à stocker des données aux structures variables.

1.3 Justification des choix technologiques

1.3.1 Sécurité et Authentification : Firebase

L'un des piliers de notre application est la sécurité. Nous avons choisi Firebase Authentication pour gérer l'intégralité du cycle de vie des comptes utilisateurs. Ce choix est motivé par la nécessité d'avoir un système d'authentification fiable (gérant les mots de passe, les emails de vérification et la récupération de compte) sans avoir à gérer la complexité du hachage des mots de passe côté serveur. L'intégration de Firebase Admin SDK sur notre backend Node.js permet une validation parfaite des tokens de session et garantit qu'un utilisateur ne peut accéder qu'à ses propres données.

1.3.2 Le Backend : Node.js et l'API REST

Node.js a été choisi pour son moteur V8 extrêmement performant et sa capacité à gérer des requêtes non-bloquantes. Cela permet à l'application de rester réactive même lors de fortes charges. Express.js complète ce choix en fournissant une structure robuste pour définir nos routes API.

1.3.3 Base de données NoSQL : MongoDB

Dans une application de recettes, le nombre d'ingrédients ou d'étapes varie d'un plat à l'autre. Le modèle Document de MongoDB est idéal ici, car il nous évite de créer des tables relationnelles trop complexes avec de multiples jointures. MongoDB Compass a été utilisé pour le monitoring et la visualisation en temps réel de nos collections.

1.3.4 Le Client Mobile : React Native

Le choix de React Native permet une base de code unique pour Android et iOS, réduisant ainsi les coûts et le temps de développement tout en offrant des performances proches du natif grâce au pont JavaScript.

1.4 Conclusion du chapitre

L'étude technique a permis de valider un stack technologique moderne et puissant. L'alliance entre la flexibilité de MongoDB et la sécurité de Firebase constitue un socle robuste pour notre application.

Chapitre 2

Analyse Fonctionnelle et Conception UML

2.1 Introduction du chapitre

La modélisation UML (Unified Modeling Language) est une étape fondamentale dans le cycle de développement d'une application. Elle permet de traduire les besoins fonctionnels et métiers en une architecture logique et structurée avant toute phase d'implémentation.

2.2 Identification des Acteurs et Cas d'Utilisation

Le système Mes Recettes est structuré autour de trois acteurs principaux :

— Le Visiteur : Peut consulter les recettes publiques sans authentification.

— L'Utilisateur : Peut s'authentifier, publier des recettes, les noter et gérer ses favoris.

— L'Administrateur : Responsable de la validation des recettes, de la modération et de la gestion globale du système.

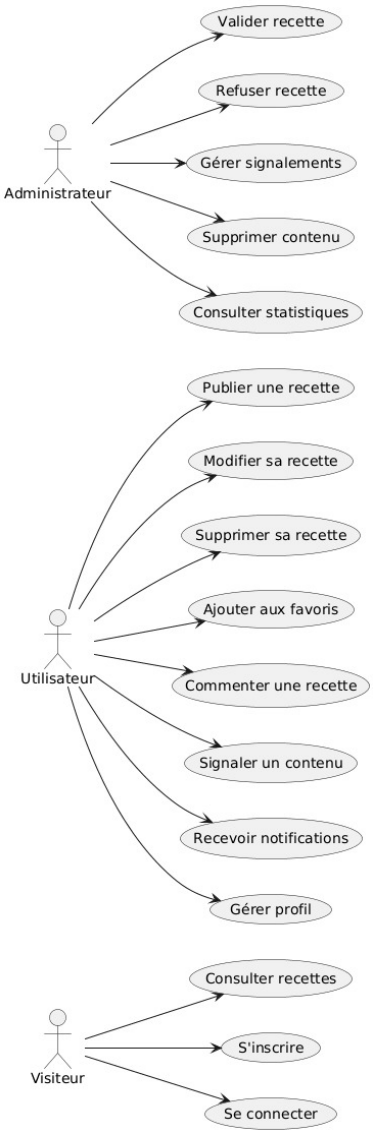


FIGURE 2.1 – Diagramme de Cas d'Utilisation de l'application Mes Recettes

2.3 Modélisation des Données (Diagramme de Classe)

La structure des données de l'application est représentée par un diagramme de classe. L'entité Recette constitue le cœur du système et est liée aux entités Utilisateur et Commentaire. La persistance des données est assurée par une base de données MongoDB.

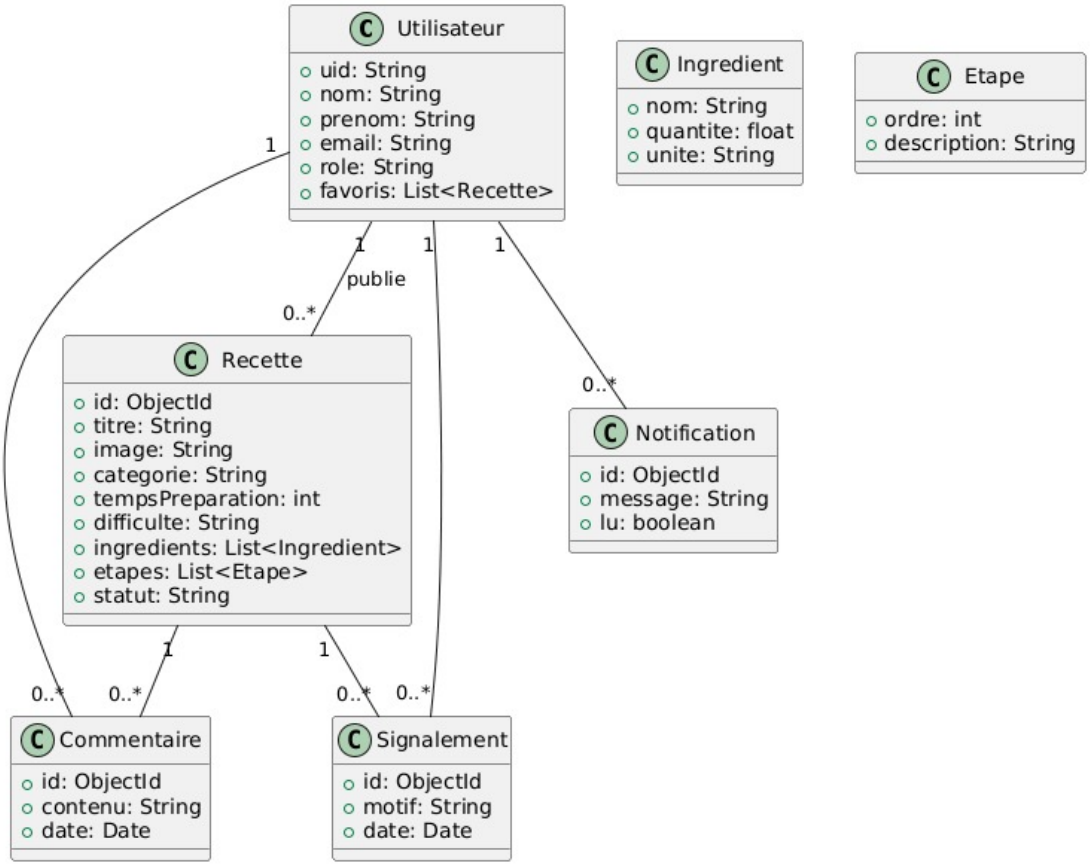


FIGURE 2.2 – Diagramme de Classe représentant la structure des données

2.4 Dynamique du Système (Diagrammes de Séquence)

Afin de mieux comprendre le comportement dynamique de l'application Mes Recettes, nous avons modélisé deux scénarios clés à l'aide de diagrammes de séquence : l'authentification de l'utilisateur et l'ajout d'une recette.

2.4.1 Diagramme de Séquence – Authentification

Ce diagramme illustre le processus d'authentification d'un utilisateur. L'utilisateur saisit ses identifiants via l'application mobile, qui communique avec le service Firebase Authentication pour vérifier la validité des informations. En cas de succès, l'accès à l'application est autorisé ; sinon, un message d'erreur est retourné.

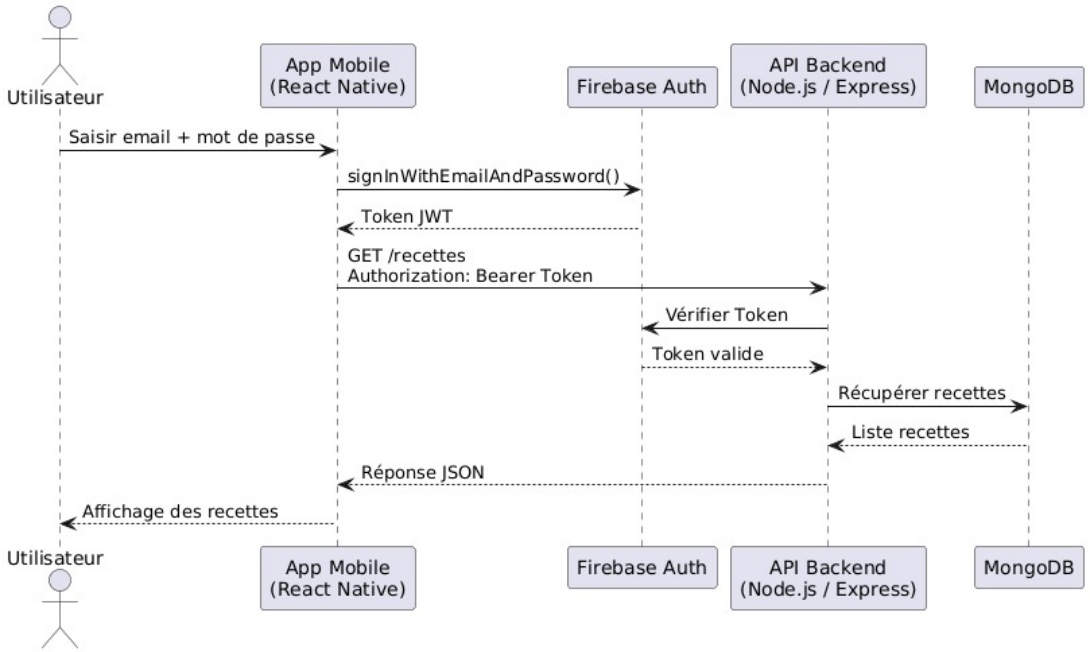


FIGURE 2.3 – Diagramme de Séquence – Authentification via Firebase

2.4.2 Diagramme de Séquence – Ajout d'une Recette

Ce diagramme décrit le scénario d'ajout d'une recette par un utilisateur authentifié. Les données saisies dans l'interface mobile sont envoyées au backend via des requêtes sécurisées. Après validation, la recette est enregistrée dans la base de données avec le statut EN ATTENTE, en attente de validation par l'administrateur.

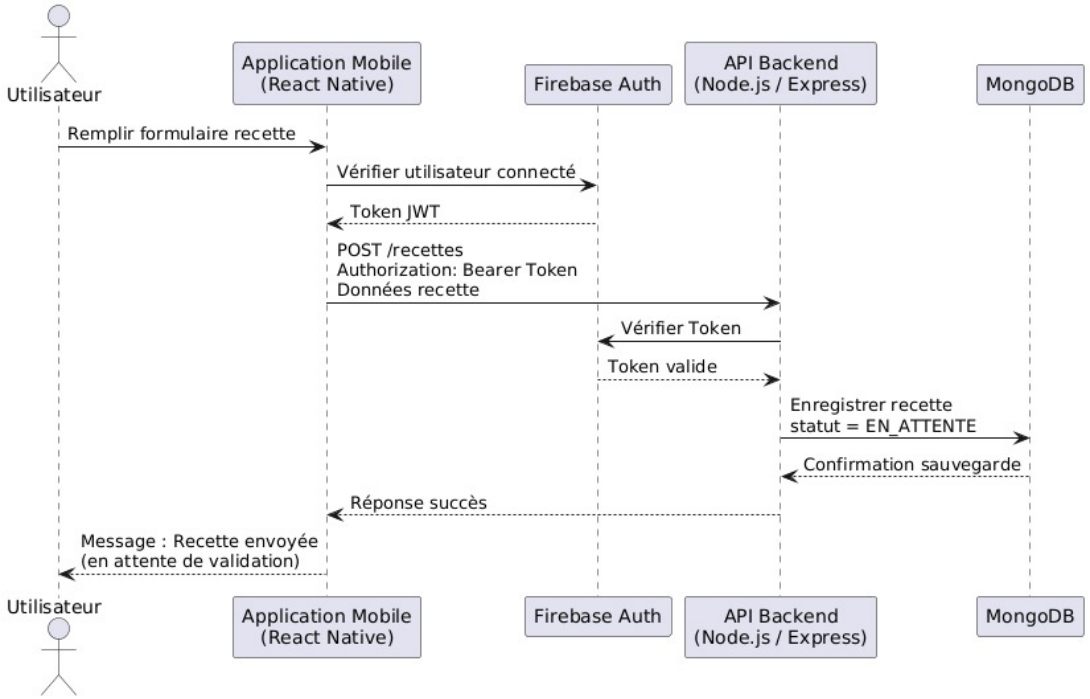


FIGURE 2.4 – Diagramme de Séquence – Ajout d'une recette

2.5 Diagramme d'Activité de l'Authentification

Le diagramme d'activité ci-dessous décrit le processus d'authentification d'un utilisateur. Il met en évidence les différentes étapes depuis la saisie des identifiants jusqu'à la validation via Firebase, avec une distinction claire entre succès et échec de connexion.

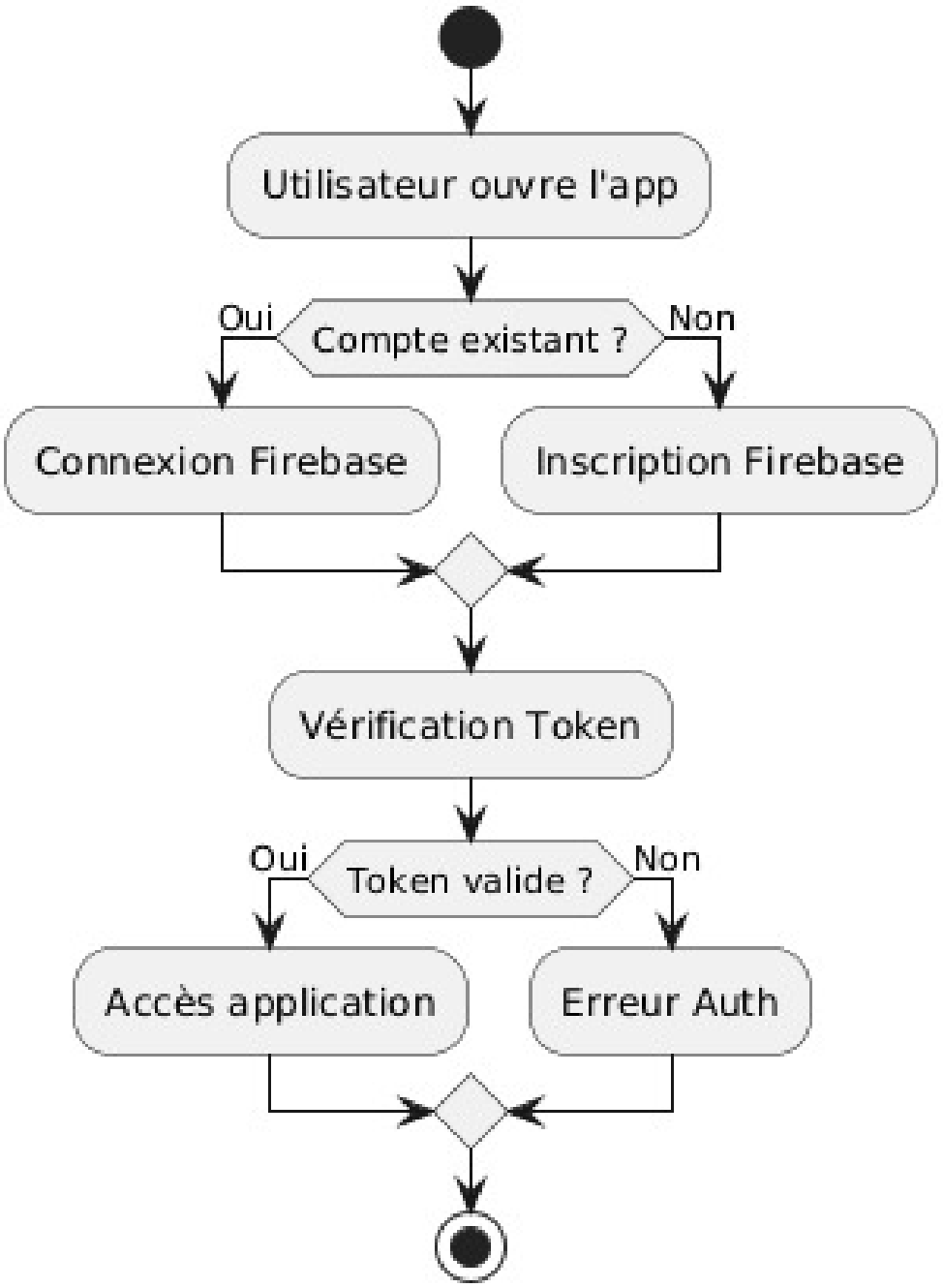


FIGURE 2.5 – Diagramme d'Activité – Processus d'authentification

2.6 Conclusion du chapitre

Cette phase de conception UML a permis de définir clairement les rôles des acteurs, la structure des données et les interactions dynamiques du système. Elle constitue une base solide pour l'implémentation du backend et du frontend de l'application Mes Recettes.

Chapitre 3

Structure du Projet et Architecture Backend

3.1 Introduction du chapitre

Un projet Full-Stack demande une organisation rigoureuse du code pour rester maintenable. Dans ce chapitre, nous explorons l'arborescence des dossiers et la logique interne du serveur Node.js.

3.2 Organisation générale du code

Le projet est divisé en deux mondes distincts qui communiquent via HTTP/REST :

3.2.1 Arborescence du Backend

- /config : Contient db.js pour la connexion MongoDB et firebaseAdmin.js. - /models : Définition des schémas Mongoose (Recette, Utilisateur, Commentaire). - /controllers : Contient la logique métier pure (ex : syncUser pour la synchro Firebase). - /routes : Les points d'entrée de l'API REST.

3.3 La Logique des Contrôleurs

Le contrôleur utilisateurController.js joue un rôle crucial. Il assure la synchronisation entre Firebase (qui gère le login) et MongoDB (qui stocke les données métier). À chaque première connexion, le système vérifie si l'utilisateur existe dans notre base et le crée si nécessaire.

3.4 Le Modèle de Données Recette

Le modèle Recette.js définit la structure complexe d'un plat. Il intègre des tableaux d'objets pour les ingrédients (un ingrédient possède un nom, une quantité et une unité) et les étapes de préparation ordonnées.

3.5 Conclusion du chapitre

L'architecture backend est conçue pour être modulaire. Cette séparation entre routes, contrôleurs et modèles permet de faire évoluer le système sans casser l'existant.

Chapitre 4

Réalisation des Interfaces et Étude des Modes Utilisateurs

4.1 Introduction du chapitre

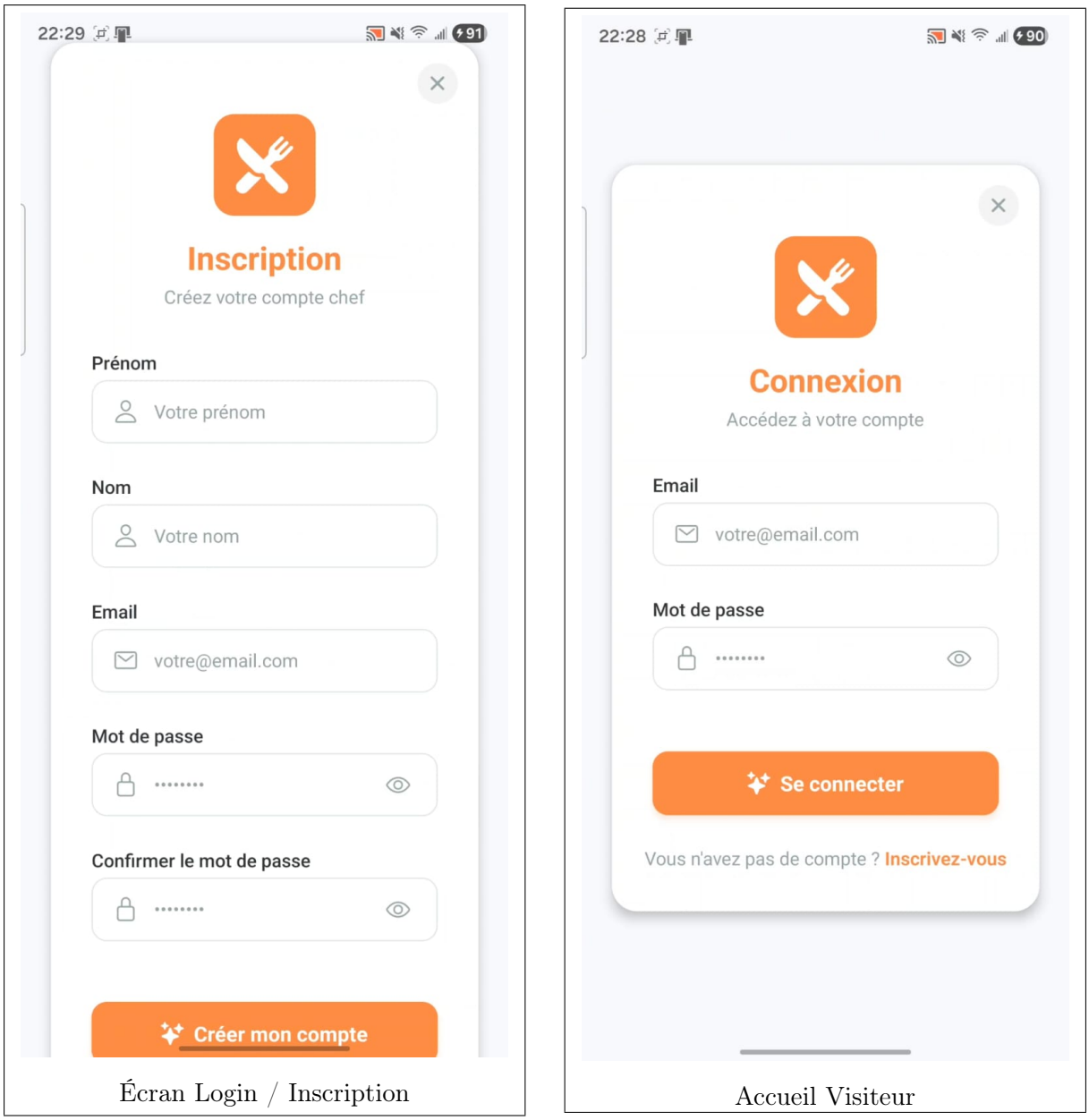
L'interface utilisateur est le point de contact direct entre la logique complexe du backend et l'expérience humaine. Pour AppliRecette, nous avons conçu une interface qui s'adapte dynamiquement selon le profil de la personne connectée. Ce chapitre détaille l'implémentation des écrans dans les trois modes principaux : Visiteur, Utilisateur Authentifié et Administrateur, tout en expliquant les fonctionnalités spécifiques à chaque rôle (Profil, Notifications, Signalements).

4.2 Le Mode Visiteur : Immersion Spontanée

Le mode visiteur est conçu pour permettre une découverte rapide de l'application sans barrière d'inscription immédiate.

— Écrans d'Accès (Login & Register) : Ces interfaces utilisent Firebase Authentication pour garantir une connexion sécurisée. Nous avons soigné le design pour que l'utilisateur puisse basculer facilement entre la connexion et l'inscription.

— Consultation des Recettes : Le visiteur peut parcourir le flux principal (Accueil.js) et voir les recettes APPROUVÉES. Cependant, les fonctionnalités d'interaction (Ajouter en favoris, Commenter) lui sont bloquées par des modales l'invitant à se connecter.



Écran Login / Inscription

Accueil Visiteur

FIGURE 4.1 – Interfaces du Mode Visiteur

4.3 Le Mode Utilisateur Authentifié : Interaction Sociale

Une fois connecté, l'utilisateur débloque l'intégralité du potentiel Full-Stack de l'application.

4.3.1 L'Écran de Profil Personnel (Profil.js)

Cet écran centralise l'identité de l'utilisateur. Il affiche ses informations récupérées de MongoDB (Nom, Prénom, Email) et permet de visualiser l'ensemble des recettes qu'il a personnellement publiées. C'est ici que l'utilisateur gère son image de marque au sein de la communauté.

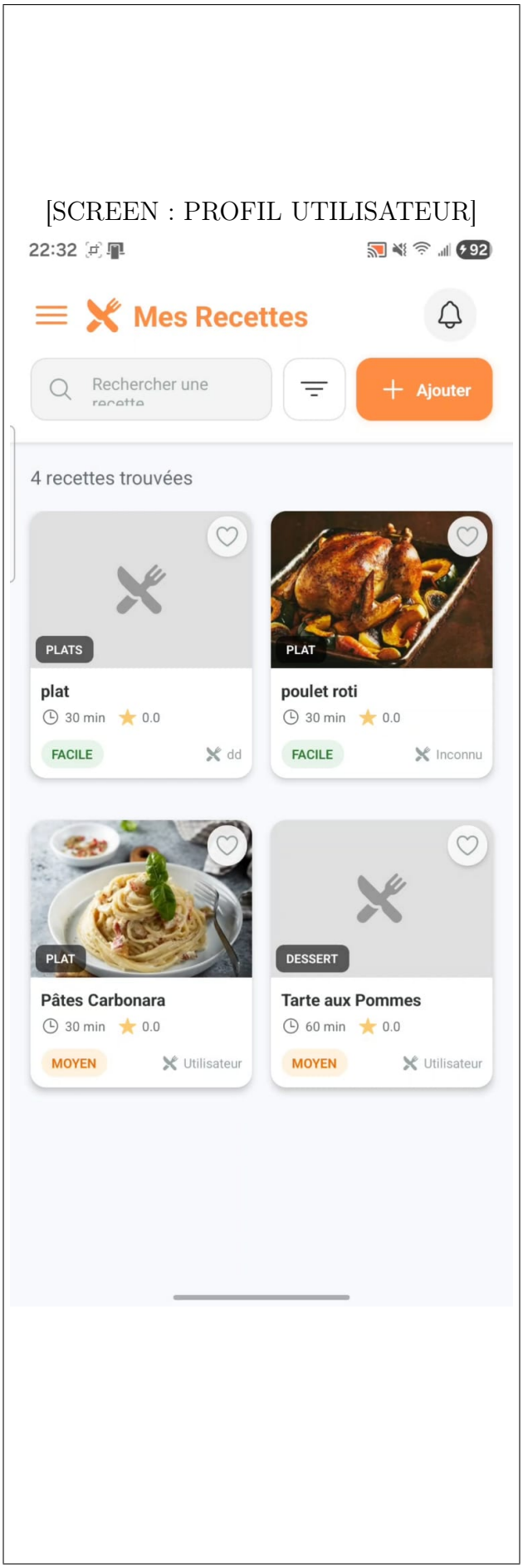


FIGURE 4.2 – Interface du Profil et Gestion Personnelle

4.3.2 Système de Notifications en Temps Réel

L'écran Notifications.js est alimenté par un contrôleur backend dédié. L'utilisateur reçoit des alertes critiques :

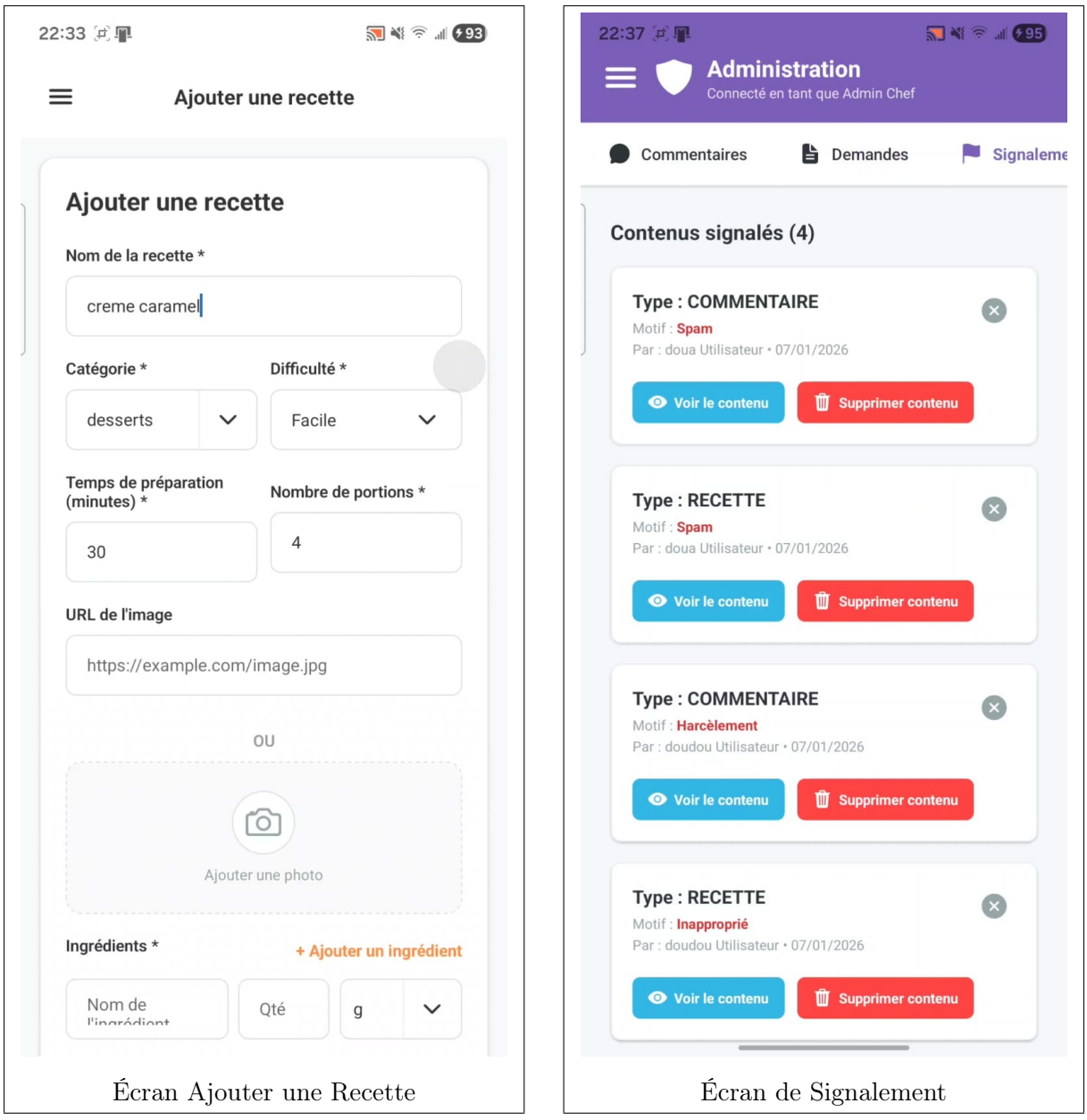
— Validation : "Votre recette [Titre] a été approuvée par l'admin".

— Engagement : "Un utilisateur a ajouté votre recette à ses favoris".

— Social : Nouveau commentaire sur l'une de ses publications.

4.3.3 Publication Dynamique et Signalements

Le formulaire de l'écran AjouterRecette.js permet une saisie complexe. L'utilisateur peut ajouter des ingrédients à la volée. En cas de contenu inapproprié sur une autre recette, il a accès à un module de Signalement qui envoie une requête POST immédiate vers la collection Signalements de MongoDB.



Écran Ajouter une Recette

Écran de Signalement

FIGURE 4.3 – Publication et Modération Communautaire

4.4 Le Mode Administrateur : Tour de Contrôle

L'administrateur dispose d'une interface spécifique (AdminDashboard.js) pour piloter l'écosystème.

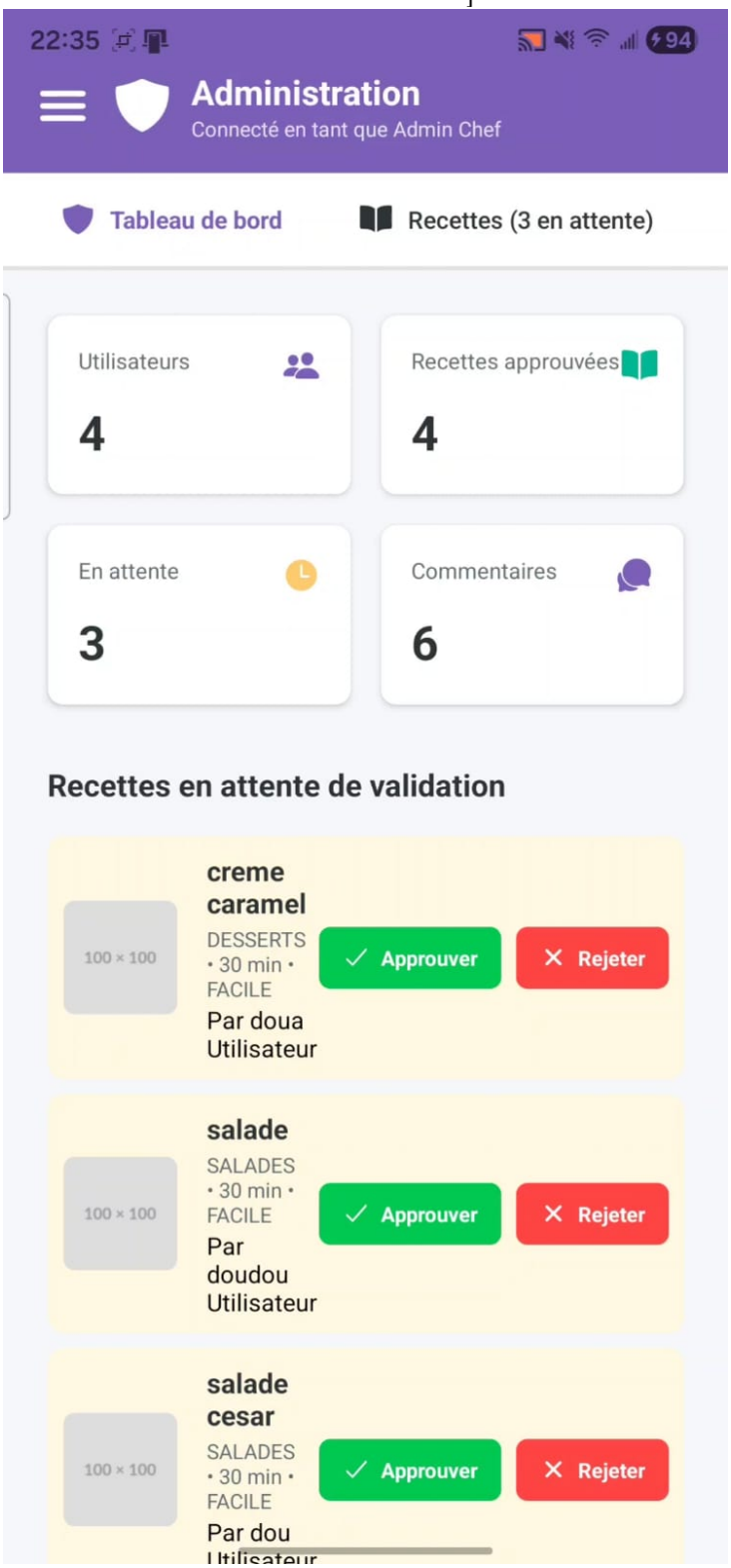
4.4.1 Dashboard et Statistiques

Grâce à des requêtes d'agrégation sur MongoDB Compass, l'admin visualise le nombre total d'utilisateurs, le volume de recettes et les statistiques de croissance de la plateforme.

4.4.2 Gestion des Signalements et Validation

L'admin reçoit une liste prioritaire des Signalements. Il peut examiner le contenu incriminé et décider de le supprimer. De même, il examine chaque recette "En Attente" pour l'approuver ou la rejeter, garantissant ainsi la qualité éditoriale de l'application.

[SCREEN : ADMIN DASHBOARD / VALIDIATION]



4.5 Conclusion du chapitre

La structuration par modes a permis d'offrir une interface claire et sécurisée. Chaque utilisateur, qu'il soit simple curieux ou administrateur rigoureux, dispose des outils nécessaires pour interagir avec la base de données de manière fluide.

Chapitre 5

Sécurité, Tests et Optimisations du Système

5.1 Introduction du chapitre

Un projet Full-Stack ne peut être considéré comme abouti sans une phase rigoureuse de sécurisation et de tests. Ce chapitre traite de la protection des données via Firebase, des tests fonctionnels de l'API et des optimisations de performance qui rendent l'application fluide.

5.2 Stratégie de Sécurité Hybride

Nous avons mis en place une sécurité à deux niveaux :

— Niveau 1 (Auth) : Firebase Authentication gère les identifiants et génère des Tokens JWT.

— Niveau 2 (Autorisation) : Côté Backend Node.js, des Middlewares personnalisés vérifient si l'utilisateur possède les droits suffisants (ex : seul un Admin peut accéder à la route /api/admin/signalements).

5.3 Tests de Fonctionnalité et Scénarios d'Erreurs

Nous avons simulé plusieurs scénarios critiques pour tester la robustesse de l'application :

— Validation des formulaires : Tentative de publication d'une recette sans titre ou sans ingrédients (bloquée par le schéma Mongoose).

— Concurrence : Plusieurs utilisateurs ajoutant simultanément la même recette en favoris (gestion des doublons dans MongoDB).

— Edge Cases : Tentative d'accès aux routes admin par un utilisateur standard (redirection automatique vers Login).

5.4 Optimisations de Performance Mobile

Pour garantir une expérience "Premium" sans ralentissements :

— FlatList Optimization : Utilisation de la mise en cache des composants pour un défilement ultra-fluide dans l'écran Accueil.

— Image Scaling : Les images de recettes sont optimisées pour réduire la consommation de données mobiles.

— MongoDB Indexing : Les recherches par catégories sont accélérées grâce à des index sur le champ categorie dans la base de données.

5.5 Conclusion du chapitre

La rigueur apportée à la sécurité et aux tests fait de AppliRecette une solution fiable et prête pour un déploiement réel. L'optimisation des ressources garantit une satisfaction utilisateur élevée, quel que soit le modèle de smartphone utilisé.

Conclusion Générale et Perspectives

La réalisation du projet AppliRecette a marqué une étape fondamentale dans notre formation d'ingénierie à l'EMSI. Ce travail de conception et de développement nous a permis de mettre en pratique l'intégralité des concepts théoriques abordés durant notre cursus de spécialisation en informatique et réseaux.

D'un point de vue technique, ce projet nous a confrontés aux réalités du développement Full-Stack. L'enjeu majeur a été de réussir la synchronisation parfaite entre l'authentification Firebase, la logique métier du serveur Node.js et la persistance des données dans MongoDB. Nous avons réussi à bâtir un système stable, capable de gérer des formulaires dynamiques complexes et d'offrir une interface d'administration temps réel via un Dashboard de statistiques.

Sur le plan fonctionnel, l'application remplit l'ensemble des objectifs fixés par le cahier des charges :

- Une consultation fluide pour les visiteurs et une interaction riche pour les utilisateurs.
- Une sécurité renforcée grâce aux middlewares de vérification des tokens.
- Une modération efficace permettant de filtrer les contenus avant leur diffusion publique.

Toutefois, ce projet n'est qu'une base solide pour des évolutions futures. Dans une perspective à moyen terme, nous envisageons d'intégrer des fonctionnalités issues de l'Intelligence Artificielle pour proposer des recommandations personnalisées basées sur les habitudes alimentaires des utilisateurs. L'ajout d'un mode Hors-ligne (Offline) pour consulter ses favoris sans connexion internet et l'intégration de tutoriels vidéo pour chaque étape de préparation sont également des pistes d'amélioration majeures.

En conclusion, AppliRecette se positionne comme un outil complet et évolutif, démontrant que la maîtrise de l'architecture Javascript hybride est un atout majeur pour répondre aux défis du développement mobile de demain.

Bibliographie

- Documentation officielle React Native : https ://reactnative.dev/
- Documentation MongoDB et Mongoose : https ://docs.mongodb.com/
- Guide de sécurité Firebase Admin SDK : https ://firebase.google.com/docs/admin